

Solutions for Minor Exam (2025)

Lecturer: Prof Minati De

Scribe

Scribed by:

1. Akarsh Gupta (2022MT61966)
2. Pratyush Sharma (2022MT61970)
3. Rahul Athipatla (2022MT11277)
4. Rohit Yuvraj Jadekar (2022MT61984)
5. Shikhar Gupta (2022MT1221925)

1 Question 1

Prove or disprove the following:

- a) *If a DFA with n states accepts infinitely many strings, then it accepts at least one string of length k such that $n \leq k \leq 2n$.*
- b) *If a DFA with n states accepts a string of length k such that $n \leq k \leq 2n$, then it accepts infinitely many strings.*

Lemma 1 (Pumping Lemma for DFAs). *Let M be a DFA with n states. For every accepted word w with $|w| \geq n$, there is a decomposition $w = xyz$ such that $|xy| \leq n$, $|y| > 0$, and for every $i \geq 0$ the word xy^iz is accepted by M .*

1.1 Part (a)

Claim 2. *If a DFA M has n states and accepts infinitely many strings, then there exists an accepted string w with length k satisfying $n \leq k < 2n$.*

Proof. Since M accepts infinitely many strings, it must accept at least one string of length greater than or equal to its number of states, n . If it did not, all accepted strings would have length less than n , and there would only be a finite number of such strings.

Let w be an accepted string of minimal length such that $|w| \geq n$. By the Pumping Lemma for DFAs, we can decompose w as $w = xyz$ such that $|xy| \leq n$, $|y| > 0$, and xy^iz is accepted for all $i \geq 0$.

Consider the string $xy^0z = xz$. Its length is $|xz| = |w| - |y|$. Since $|y| > 0$, we have $|xz| < |w|$. By our choice of w as the shortest accepted string with length at least n , it must be that $|xz| < n$.

From this, we have $|w| - |y| < n$, which implies $|w| < n + |y|$. We also know from the Pumping Lemma that $|y| \leq |xy| \leq n$. Combining these inequalities gives:

$$|w| < n + |y| \leq n + n = 2n$$

So, the length of w , let's call it k , satisfies $n \leq k < 2n$. The statement is **true**. $\square$

1.2 Part (b)

Claim 3. *If a DFA M with n states accepts a string w with $|w| = k$ and $k \geq n$, then M accepts infinitely many strings.*

Proof. Let M be a DFA with n states and let w be an accepted string with length $k \geq n$. By the Pumping Lemma for DFAs, we can decompose w as $w = xyz$ where $|xy| \leq n$, $|y| > 0$, and xy^iz is accepted by M for all integers $i \geq 0$.

Consider the set of strings $L' = \{xy^iz \mid i = 0, 1, 2, \dots\}$. Since $|y| > 0$, the length of each string xy^iz is $|x| + i|y| + |z|$, which is different for each value of i . Therefore, the set L' contains an infinite number of distinct strings. Since every string in L' is accepted by M , the DFA M accepts infinitely many strings. The statement is **true**. $\square$

2 Question 2

Consider the language $A = \{a^ib^jc^k \mid i = j \text{ or } j = k \text{ where } i, j, k \geq 0\}$.

- Prove or disprove A is regular.
- Give a context free grammar that generates language A . Is your language ambiguous? Why or why not?
- Give a description of a non-deterministic pushdown automata for the language A

(a) A is not regular

Proof. Assume for contradiction that A is regular. Let p be the pumping length given by the pumping lemma for regular languages. Consider

$$s = a^pb^p \in A$$

(since $i = p$, $j = p$, $k = 0$ so $i = j$). By the pumping lemma write $s = xyz$ with $|xy| \leq p$, $|y| > 0$ and $xy^tz \in A$ for all $t \geq 0$.

Because $|xy| \leq p$ and the prefix of length p of s consists entirely of a 's, the substring y is of the form a^ℓ with $1 \leq \ell \leq p$. Take $t = 0$. Then

$$xy^0z = a^{p-\ell}b^p.$$

In this string the numbers of a 's and b 's are $p - \ell$ and p respectively, so $i' = p - \ell \neq p = j$. Also $k = 0$, hence $j \neq k$. Thus neither $i' = j$ nor $j = k$ holds, so $xy^0z \notin A$, contradicting the pumping lemma. Hence A is not regular. $\square$

(b) A CFG for A ; ambiguity

Write $A = L_1 \cup L_2$ where

$$L_1 = \{a^n b^n c^m \mid n, m \geq 0\}, \quad L_2 = \{a^m b^n c^n \mid m, n \geq 0\}.$$

Define grammar $G = (V, \Sigma, P, S)$ with

$$V = \{S, T, U, V', W\}, \quad \Sigma = \{a, b, c\},$$

and productions

$$\begin{aligned} S &\rightarrow TU \mid V'W, \\ T &\rightarrow aTb \mid \lambda, \\ U &\rightarrow cU \mid \lambda, \\ V' &\rightarrow aV' \mid \lambda, \\ W &\rightarrow bWc \mid \lambda. \end{aligned}$$

Claim: $L(G) = A$.

Proof: (1) $\subseteq$: Any derivation from S uses either $S \rightarrow TU$ or $S \rightarrow V'W$. If $S \Rightarrow TU$ then T yields $a^n b^n$ and U yields c^m , so the yield is $a^n b^n c^m \in L_1 \subseteq A$. If $S \Rightarrow V'W$ then V' yields a^m and W yields $b^n c^n$, so the yield is $a^m b^n c^n \in L_2 \subseteq A$. Hence $L(G) \subseteq A$.

(2) $\supseteq$: Every string in A lies in L_1 or L_2 . For any $a^n b^n c^m$ use $S \rightarrow TU$ with $T \Rightarrow a^n b^n$, $U \Rightarrow c^m$. For any $a^m b^n c^n$ use $S \rightarrow V'W$ with $V' \Rightarrow a^m$, $W \Rightarrow b^n c^n$. Thus $A \subseteq L(G)$. $\square$

Claim: Grammar G is ambiguous.

Proof. For each $n \geq 0$ consider $w_n = a^n b^n c^n$. Then

$$S \Rightarrow TU \Rightarrow (a^n b^n)(c^n)$$

is a leftmost derivation (use $T \Rightarrow a^n b^n$, $U \Rightarrow c^n$), and

$$S \Rightarrow V'W \Rightarrow (a^n)(b^n c^n)$$

is a different leftmost derivation (use $V' \Rightarrow a^n$, $W \Rightarrow b^n c^n$). These two leftmost derivations produce distinct parse trees (the root production of S is different), so w_n has at least two distinct parse trees for every n . Hence G is ambiguous. $\square$

(c) A nondeterministic PDA for A

We give an NPDA M that accepts by final state and (i) enforces the form $a^* b^* c^*$ and (ii) nondeterministically chooses to verify either $i = j$ or $j = k$.

Formal definition

Let

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

where

$$\begin{aligned} Q &= \{q_0, q_{1a}, q_{1b}, q_{1c}, q_{2a}, q_{2b}, q_{2c}, q_{acc}\}, \\ \Sigma &= \{a, b, c\}, \quad \Gamma = \{Z_0, A\}, \quad F = \{q_{acc}\}. \end{aligned}$$

Intuition: from q_0 nondeterministically choose branch 1 (check $i = j$) or branch 2 (check $j = k$). Each branch enforces three phases (read a 's, then b 's, then c 's) so input must be in $a^*b^*c^*$.

We list all transitions $\delta(q, \sigma, X) \ni (q', \gamma)$ (where γ is the string replacing the top stack symbol).

Initial choice

$$\delta(q_0, \lambda, Z_0) \ni (q_{1a}, Z_0), \quad \delta(q_0, \lambda, Z_0) \ni (q_{2a}, Z_0).$$

Branch 1 (verify $i = j$) — states q_{1a} (reading a 's), q_{1b} (reading b 's), q_{1c} (reading c 's).

$$\begin{aligned} &(\text{push one } A \text{ per } a) \quad \forall X \in \{Z_0, A\} : \delta(q_{1a}, a, X) \ni (q_{1a}, AX). \\ &(\text{on first } b: \text{ enter } b\text{-phase and pop one } A) \quad \delta(q_{1a}, b, A) \ni (q_{1b}, \lambda). \\ &(\text{no transition on } b \text{ when top is } Z_0) \\ &(\text{continue popping one } A \text{ per } b) \quad \delta(q_{1b}, b, A) \ni (q_{1b}, \lambda). \\ &(\text{when all } A\text{'s popped, move to } c\text{-phase}) \quad \delta(q_{1b}, \lambda, Z_0) \ni (q_{1c}, Z_0). \\ &(\text{allow zero } a\text{'s and zero } b\text{'s}) \quad \delta(q_{1a}, \lambda, Z_0) \ni (q_{1c}, Z_0). \\ &(\text{read remaining } c\text{'s only}) \quad \delta(q_{1c}, c, Z_0) \ni (q_{1c}, Z_0). \\ &(\text{accept when stack is } Z_0 \text{ and input exhausted}) \quad \delta(q_{1c}, \lambda, Z_0) \ni (q_{acc}, Z_0). \end{aligned}$$

Branch 2 (verify $j = k$) — states q_{2a} (reading a 's), q_{2b} (reading b 's), q_{2c} (reading c 's).

$$\begin{aligned} &(\text{stay in } q_{2a} \text{ while reading } a\text{'s; do not change stack}) \quad \forall X \in \{Z_0, A\} : \delta(q_{2a}, a, X) \ni (q_{2a}, X). \\ &(\text{on first } b: \text{ enter } b\text{-phase and push one } A) \quad \forall X \in \{Z_0, A\} : \delta(q_{2a}, b, X) \ni (q_{2b}, AX). \\ &(\text{continue pushing one } A \text{ per } b) \quad \forall X \in \{Z_0, A\} : \delta(q_{2b}, b, X) \ni (q_{2b}, AX). \\ &(\text{on first } c: \text{ enter } c\text{-phase and pop one } A) \quad \delta(q_{2b}, c, A) \ni (q_{2c}, \lambda). \\ &(\text{continue popping one } A \text{ per } c) \quad \delta(q_{2c}, c, A) \ni (q_{2c}, \lambda). \\ &(\text{after all } A\text{'s popped, accept if input exhausted}) \quad \delta(q_{2c}, \lambda, Z_0) \ni (q_{acc}, Z_0). \end{aligned}$$

No other transitions are defined. Undefined moves are rejecting.

Correctness (sketch)

- **Structure enforcement:** Each branch has three phases and transitions that only allow symbols in the correct phase order $a^*b^*c^*$. Any input not of this form is rejected.

- **Branch 1 correctness:** The machine pushes one A per a and pops one A per b . The stack is empty (down to Z_0) after the b -block if and only if the number of a 's equals the number of b 's.
- **Branch 2 correctness:** The machine pushes one A per b and pops one A per c . Acceptance occurs if and only if the number of b 's equals the number of c 's.
- Since M nondeterministically chooses a branch and accepts if any branch accepts, $L(M) = \{a^i b^j c^k \mid i = j \text{ or } j = k\} = A$.

3 Question 3

Show that the language $B = \{b^n \mid n \text{ is not a square}\}$ is not regular.

Proof by contradiction. We prove that the complement, $\bar{B} = \{b^n \mid n \text{ is a perfect square}\}$, is not regular. Since regular languages are closed under complement, this implies B is not regular.

Assume $\bar{B}$ is regular and let p be its pumping length. Choose the string $s = b^{p^2} \in \bar{B}$. Since $|s| = p^2 \geq p$, we can apply the Pumping Lemma. Decompose $s = xyz$ such that $|xy| \leq p$, $|y| \geq 1$, and $xy^i z \in \bar{B}$ for all $i \geq 0$.

Let $|y| = k$, where $1 \leq k \leq p$. Consider the string $w' = xy^2 z$. Its length is $|w'| = |s| + |y| = p^2 + k$. For w' to be in $\bar{B}$, its length must be a perfect square. However, we have the inequality:

$$p^2 < p^2 + k \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2$$

The length $p^2 + k$ is strictly between two consecutive squares, p^2 and $(p+1)^2$, so it cannot be a square. This means $xy^2 z \notin \bar{B}$, which is a contradiction. Therefore, $\bar{B}$ is not regular, and consequently, B is not regular. $\square$

4 Question 4

Prove that $\bar{C}$ (the complement of C in Σ^* with $\Sigma = \{a, b\}$) and C^* (the Kleene star of C) are context-free.

The complement $\bar{C}$ is context-free

Proof. A string $w \in \Sigma^*$ fails to belong to C exactly in one of the following mutually exclusive ways:

1. w contains the substring **ba** (i.e. some occurrence of a b before a later a), so $w \notin a^* b^*$.
2. w is of the form $a^m b^n$ (that is, $w \in a^* b^*$) but $m \neq n$.

Therefore

$$\bar{C} = L_1 \cup L_2 \cup L_3,$$

where

$$\begin{aligned} L_1 &= \{ w \in \Sigma^* : w \text{ contains the substring } ba \} = \Sigma^* ba \Sigma^*, \\ L_2 &= \{ a^m b^n : m > n \}, \\ L_3 &= \{ a^m b^n : m < n \}. \end{aligned}$$

We now argue each of L_1, L_2, L_3 is context-free.

(i) L_1 is regular (hence CFL). The language of strings containing a fixed substring (here ba) is regular:

$$L_1 = \Sigma^* ba \Sigma^*.$$

Regular languages are context-free, so L_1 is a CFL.

(ii) $L_2 = \{ a^m b^n : m > n \}$ is context-free. We give a CFG $G_2 = (\{S, A\}, \{a, b\}, P_2, S)$ that generates L_2 . Productions P_2 :

$$\begin{aligned} S &\rightarrow aSb \mid A, \\ A &\rightarrow aA \mid a. \end{aligned}$$

Intuition: each use of $S \rightarrow aSb$ pairs one leading a with one trailing b ; when the recursion stops at A , A produces at least one extra a . Thus any derived string has the form $a^{t+r}b^t$ with $t \geq 0$ and $r \geq 1$, i.e. $m > n$. Conversely, any $a^m b^n$ with $m > n$ can be written $a^{t+r}b^t$ with $r \geq 1, t \geq 0$, and is derived by applying $S \rightarrow (aSb)^t A$ and then expanding A to produce r a 's. Hence $L(G_2) = L_2$.

(iii) $L_3 = \{ a^m b^n : m < n \}$ is context-free. Symmetrically define CFG $G_3 = (\{S, B\}, \{a, b\}, P_3, S)$ with productions

$$\begin{aligned} S &\rightarrow aSb \mid B, \\ B &\rightarrow Bb \mid b. \end{aligned}$$

Here B produces at least one b ; using aSb pairs off equal numbers of outer a 's and b 's before finally emitting extra b 's via B . By the same argument as for G_2 , $L(G_3) = L_3$.

Conclusion. Each L_1, L_2, L_3 is context-free. Since CFLs are closed under finite union, $\bar{C} = L_1 \cup L_2 \cup L_3$ is a context-free language. This completes the proof. $\square$

The Kleene star C^* is context-free

Theorem 4. Let $C = \{ a^n b^n : n \geq 0 \}$. Then C^* is context-free.

Proof. By definition,

$$C^* = \{ w_1 w_2 \cdots w_k : k \geq 0, w_i \in C \text{ for each } i \}.$$

Equivalently, strings in C^* are concatenations of zero or more blocks, where each block is of the form $a^n b^n$ (and different blocks may have different n 's). We give a CFG that generates exactly these concatenations.

Consider the grammar $G = (\{S\}, \{a, b\}, P, S)$ with productions

$$\begin{array}{lcl} S & \rightarrow & SS \mid T \mid \varepsilon. \\ T & \rightarrow & aTb \mid \varepsilon \end{array}$$

The first rule makes any number of concatenations, while the second rule makes words in language C . $\square$